

**LAPORAN PRAKTIKUM
PEMROGRAMAN BERBASIS FRAMEWORK**

WEEK 13: Middleware & Route Protection



Disusun Oleh :
Ghoffar Abdul Ja'far 2341720035/TI3B

**JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI MALANG
2025/2026**

JOBSHEET PRAKTIKUM

Mata Kuliah: Pemrograman Framework

Pertemuan: Middleware & Route Protection

Topik: Proteksi Halaman dengan Middleware

A. Capaian Pembelajaran (CPMK)

Mahasiswa mampu:

1. Menjelaskan konsep Middleware pada Next.js.
 2. Membedakan redirect menggunakan useEffect dan Middleware.
 3. Membuat file middleware.ts.
 4. Mengatur proteksi route tertentu.
 5. Mengimplementasikan sistem login sederhana menggunakan Middleware.
-

B. Dasar Teori

1 Apa itu Middleware?

Middleware adalah kode yang dijalankan sebelum request halaman diproses.

Alur kerja:

User akses halaman



Middleware dijalankan



Cek kondisi (login / role / dll)



Lanjutkan request atau redirect

- Middleware berjalan di server/edge layer sebelum halaman dirender.
-

2 Mengapa Tidak Menggunakan useEffect?

Redirect dengan useEffect:

```
tsx

useEffect(() => {
  if (!isLogin) {
    router.push('/login')
  }
}, [])
```

Masalah:

- Halaman sempat terbuka dulu
- Terjadi "glitch"
- Kurang aman

Middleware:

- Redirect sebelum halaman dirender
- Tidak ada glitch
- Lebih secure

✂ C. Langkah Praktikum

◆ Bagian 1 – Membuat Middleware

- Modifikasi file index.tsx pada folder src/pages/produk

```
9  const ProdukPage = () => {
10    // const [isLogin, setIsLogin] = useState(false);
11    const { push } = useRouter(); // Remove this useless
12    const [products, setProducts] = useState([]); // Remove this
13    // console.log("products:" + products); // Remove this
19    const [isRefreshing, setIsRefreshing] = useState(false);
20  }
```

- Buat file: src/middleware.ts Seajar dengan folder pages.



◆ Bagian 2 – Struktur Dasar Middleware

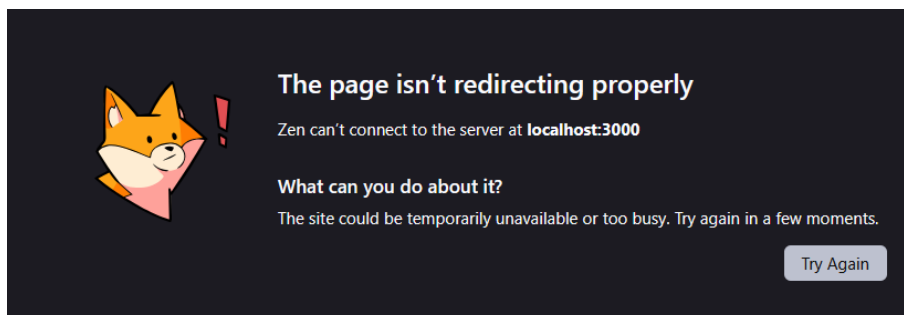
```
src > TS middleware.ts > middleware
1 import { NextResponse } from 'next/server'
2 import type { NextRequest } from 'next/server'
3
4 export function middleware(request: NextRequest) {
5   return NextResponse.next();
6 }
```

- Jika menggunakan NextResponse.next() → tidak ada redirect.
- Jadi masih bisa mengakses ke <http://localhost:3000/produk>

◆ Bagian 3 – Redirect Sederhana

```
src > TS middleware.ts > middleware
1 import { NextResponse } from 'next/server'
2 import type { NextRequest } from 'next/server'
3
4 export function middleware(request: NextRequest) {
5   return NextResponse.redirect(new URL('/', request.url));
6 }
7 // return NextResponse.next(); Remove this commented out code.
8 }
```

- Semua halaman akan redirect ke home dan error dikarenakan terus menerus loading



◆ Bagian 4 – Batasi Route Tertentu

- Untuk mengatasi pada bagian 3 maka perlu pembatasan route

```
export const config = {
  matcher: ["/produk", "/about"],
}
```

- Artinya:
 - Middleware hanya berlaku untuk /products dan /about
 - Halaman lain tetap normal
 - Ketika user mengakses halaman produk dan about maka akan langsung redirect ke halaman home

◆ Bagian 5 – Simulasi Sistem Login

- Modifikasi file middleware.ts

```
export function middleware(request: NextRequest) {
  const isLogin = false;
  if (isLogin) {
    return NextResponse.next();
  } else {
    return NextResponse.redirect(new URL("/auth/login", request.url));
  }
}

// return NextResponse.redirect(new URL('/', request.url)); Remove this commented out
// return NextResponse.next(); Remove this commented out code.
}

export const config = {
  matcher: ["/produk/:path*", "/about"],
};
```

- Jika user langsung mengakses ke alamat <http://localhost:3000/produk> tidak akan bisa user akan diarahkan ke halaman login

D. Pengujian

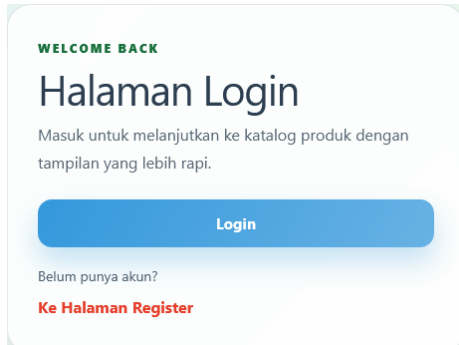
- ✎ Uji 1 – isLogin = false

Akses:

/products

Hasil:

➡ Redirect ke /login



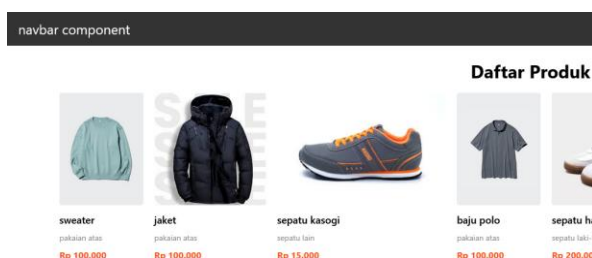
- ✎ Uji 2 – isLogin = true

Ubah:

const isLogin = true

Hasil:

➡ Bisa mengakses /products



❧ Uji 3 – Tambahkan Multiple Route

```
export const config = {  
  matcher: ['/products', '/about']  
}
```

Sekarang:

- /products dan /about butuh login
- Halaman lain bebas

navbar component

Profile Page
edit

Footer component

🚩 E. Perbandingan Middleware vs useEffect

Aspek	useEffect	Middleware
Redirect timing	Setelah render	Sebelum render
Glitch	Ada	Tidak
Security	Lemah	Lebih aman
Skalabilitas	Harus tiap halaman	Sekali di middleware



G. Pertanyaan Analisis

1. Mengapa middleware lebih aman dibanding useEffect?
= Berjalan di server sebelum halaman dirender di browser.
2. Mengapa middleware tidak menimbulkan glitch?
= Karena redirect terjadi sebelum browser merender tampilan.
3. Apa risiko jika semua halaman diproteksi tanpa pengecualian?
= Terjadi infinite redirect loop pada halaman login.
4. Kapan middleware tidak diperlukan?
= Saat mengakses halaman publik tanpa batasan akses.
5. Apa perbedaan middleware dan API route?
= Middleware intersep request, API route memproses dan merespons.